

ShopAI — Technical Overview

Multi-vendor e-commerce marketplace prototype · Status as of commit `c456fd5` (2026-10-05)

1. What it is

ShopAI is a working prototype of an Indian (INR) multi-vendor marketplace with two sides:

- **Customers:** browse stores and products, add to cart, check out with simulated UPI / Card / COD, track orders on a 6-stage timeline (Placed → Confirmed → Packed → Shipped → Out for Delivery → Delivered), request returns, and chat with an AI shopping assistant.
- **Admin / merchant:** a Shopify-style Store Dashboard (overview, products with image upload, inventory, orders, customers, returns, settings) and a 5-step store onboarding wizard.

The whole app is behind login. New registrations get the `customer` role; the seeded `admin` account manages every store.

Size: roughly 9k lines of hand-written JS/JSX plus a 2k-line seed dataset (`src/services/initialData.js`).

2. Frontend (`src/`)

- **Stack:** React 19, Vite 8, lucide-react, canvas-confetti, plain CSS (`src/index.css`), oxlint. No TypeScript.
- **Routing:** no react-router. `src/App.jsx` keeps the path and query string in state, uses `pushState` / `popstate`, and picks the page with an `if` chain. The login gate and admin-only dashboard check live there.
- **Services:** one module per entity in `src/services/`, each method one API call through `apiFetch` (`src/services/api.js`). It attaches the session token, and on a 401 clears the session and reloads to the login screen. For a same-origin API (`/api`, Azure Static Web Apps) it sends the token as `X-Session-Token`, because the platform reserves `Authorization`.
- **Browser-only state** (`localStorage`): cart, session, selected dashboard store. Everything else is server-side.

3. Backend (`backend/src/`)

One Lambda-style handler (`backend/src/index.mjs`) serves every route. The same handler runs on AWS Lambda, on Azure Functions (through an adapter) and locally.

Routing and auth

- Route table of `[method, pattern, handler, access]`, where access is `public`, `user` or `admin`. Returns 404/405 for unknown routes, caps request bodies at 256 KB, and returns a generic 500 for unexpected errors.
- Passwords: PBKDF2-SHA256, 100k iterations, constant-time comparison.
- Sessions: HS256 JWT signed with `node:crypto`, 12-hour lifetime. The token header is ignored (always HS256), which blocks `alg` confusion.
- Users are keyed by email; duplicate registration is blocked with a conditional put.

Data model

Five DynamoDB tables, each with a single hash key: `users` (email), `stores`, `products`, `orders`, `returns` (id). Lookups are full-table scans filtered in the handler; secondary indexes are a planned follow-up.

Checkout (`orders.mjs` → `createOrders`)

- Prices are read from the database, never taken from the client.
- The cart is split into one order per store. Shipping (₹99, free above ₹1,500) and discount (10% above ₹3,000) are shared out by each store's portion of the subtotal; the last order takes the rounding remainder.
- One `TransactWriteItems` writes the orders, decrements stock (conditional on stock being unchanged since it was read) and updates store metrics. On a conflict it retries up to 3 times, then returns 409.
- Status changes use a conditional put on the previous status, so side effects run exactly once: cancelling restocks and reverses the sale; a return restocks once and refunds once (tracked by `restocked` / `refunded` flags).
- Orders and returns are visible only to their owner or the admin. Returns are allowed only on your own delivered orders, with no duplicate open return per item.

AI shopping assistant (`assistant.mjs`)

- Calls Azure OpenAI chat completions directly (API version `2024-10-21`). Endpoint, key and deployment come from env vars; it returns 503 if they are missing.
- Grounding: up to 200 trimmed product rows are placed in the system prompt as JSON. No RAG or embeddings.
- Limits: last 10 messages, 2,000 characters per message, 600 output tokens, temperature 0.4, JSON response mode, 20-second timeout (504).
- Output validation: a content-filter block gets a canned reply; malformed JSON is an error; `productIds` are kept only if they exist in the catalog, deduplicated and capped at 4. A 429 from Azure is passed through.

4. Cloud services

The repo has two deployment targets. Azure OpenAI is used by both.

AWS — SAM stack in `backend/template.yaml` (region `ap-south-1`)

Service	What it is used for
Lambda	One Node 22 function on arm64 (512 MB, 25 s timeout) handling every API route.
API Gateway (HTTP API)	Routes all requests (<code>ANY /{proxy+}</code>) to the Lambda, handles CORS, throttles at 10 req/s (burst 20).
DynamoDB	The 5 tables, pay-per-request; transactions used at checkout.
S3 — images bucket	Product images. The browser uploads directly with a 5-minute signed URL that locks file type and size (5 MB max). <code>products/*</code> is publicly readable.
S3 — site bucket	The built frontend. By default served as a public S3 website over HTTP.
CloudFront	Optional (<code>CloudFrontEnabled=true</code>): HTTPS in front of a private site bucket. Off until AWS verifies the account.
IAM	SAM-generated permissions: Lambda gets CRUD on the 5 tables and write access to the images bucket.
CloudFormation / SAM	Creates and updates everything (<code>sam deploy</code>). <code>scripts/deploy-web.mjs</code> reads stack outputs from it.

Secrets (JWT secret, Azure OpenAI key) are passed as Lambda environment variables; Secrets Manager is not used yet.

Azure — `backend/azure.mjs`, `scripts/deploy-azure.mjs`

Service	What it is used for	Setup
Azure OpenAI	The shopping assistant (model named in the README: <code>gpt-4.1-mini</code>).	Both
Static Web Apps	Hosts the frontend; unknown paths fall back to <code>index.html</code> (<code>public/staticwebapp.config.json</code>).	Azure
SWA managed API (Azure Functions v4, Node 22)	Runs the same backend handler under <code>/api/*</code> , translating Functions requests into API Gateway events.	Azure
Blob Storage — <code>data</code> container	A full JSON copy of the database (<code>shopai-data.json</code>), rewritten after every change and loaded back on startup.	Azure
Blob Storage — <code>images</code> container	Product images, uploaded with a 5-minute write-only SAS URL and served publicly.	Azure

Deployment uses the `az` CLI and `@azure/static-web-apps-cli`; defaults are app `shopai-web` in resource group `shopai-rg`.

The Azure setup does not use AWS. The backend still talks to a DynamoDB-style database through the AWS SDK, but on Azure it points at **dynalite**, an in-memory DynamoDB emulator running in the same process. Data survives restarts only through the Blob Storage copy. `dynalite` has no transactions, so `azure.mjs` emulates them (items applied one at a time under a lock, rolled back on failure).

Local development

`backend/local.mjs` runs the handler on port 3001 against DynamoDB Local on port 8000. No cloud account needed; the assistant and image upload are unavailable locally.

5. Tests

- Backend: `backend/test/backend.test.mjs` and `azure.test.mjs` (`node --test`, no cloud account needed).
- Frontend: no tests.

6. Known gaps and risks

- **Docs lag the code.** `README.md` and `docs/handoff.md` still describe AWS as the primary deployment; the last two commits moved hosting to Azure Static Web Apps.
- **Azure data copy is last-write-wins.** If two Functions instances run at once, one can overwrite the other's changes. Every write also re-reads and re-uploads the whole database.
- **No upload size limit on Azure.** A SAS URL cannot cap size, so the 5 MB limit relies on the size the browser declares (admin-only route). On AWS, S3 enforces it.
- **Secrets in plain env vars / app settings** (JWT secret, Azure key); no Key Vault or Secrets Manager.
- **Full-table scans** for orders and returns, and the assistant scans all products on every message.
- **Any signed-in user can create a store** (`POST /stores` requires only `user`), while only the admin can manage it.
- **Payments are simulated;** routing is hand-written rather than a router library.